Promptware

Software made of prompts

by Niels Bosma (niels@ivy.app)

Lightning Talk

How can I make self-improving software?

The result of a sleepless night.

Not necessarily agentic — but often agentic.

The Problem

You write an agent prompt...

It works **okay**...

But it never gets **better**.

Every session starts from zero.

What if prompts were more like software?

Software has **source code** that evolves over time

Software has **memory** — databases, config, state

Software has determinisitic **tools** — libraries, utilities, scripts

Software has **logs** — observability, audit trails

What if your agent prompts had all of that?

Promptware: The Anatomy

Every agentic module follows the same structure:

```
MyAgent/
  Program.md     -- The "source code" (evolves over time)
  Memory/        -- Persistent knowledge across sessions
  Tools/         -- Scripts the agent creates for itself
  Logs/          -- Execution history
```

Plus a shared **Firmware** layer — a template prompt injected into every agent that bootstraps the whole thing.

```
Launcher.ps1  ⟶  Firmware.md  ⟶  Program.md  ⟶  Do the work  ⟶  Reflect
     ^                                                                |
     |________________________________________________________________|
                             next run is better
```

Powered by **Claude Code** in yolo mode (`--dangerously-skip-permissions`)

Firmware: The Bootstrap

The firmware is a **fixed template** that every agent receives. It tells the agent:

1. **Read** your `Program.md` — your instructions
2. **List** your tools and memory — your accumulated knowledge
3. **Execute** the task
4. **Reflect** — update Program.md, Memory, or Tools

The firmware **never changes**. It's the bootloader.

The Program.md **changes over time**. It's the evolving intelligence.

The Reflection Loop

At the end of **every** execution, the agent asks itself:

- Should I add new instructions to `Program.md`?
- Did I learn something worth saving to `Memory/`?
- Should I create a reusable `Tool` for next time?
- Is any of my existing knowledge outdated?

How I use this at my work on Ivy

I run ~20 promptware's built on this pattern as part of our general harness engineering system:

Plan lifecycle

- MakePlan
- UpdatePlan
- ExpandPlan
- BuildPlan
- TestPlan

Git & review

- CreateCommit
- CreatePullRequest
- MakePRs
- Sync

Each one started as a simple `Program.md`.

Each one is now full of learned patterns, self-made tools, and accumulated memory with reference to my specific systems.

Example: Meeting Prep Agent

Meeting with Göran Sandahl

12:00 • Luncha Göran <> Niels

Name: Göran Sandahl

LinkedIn: linkedin.com/in/goransandahl

Twitter: @gsandahl

Instagram: Not found

Website: gsandahl.github.io (minimal)

GitHub: github.com/gsandahl (7 repos; bio: “Smiling technologist”; most notable: simple-mcp-agent)

Reddit: Not found

Programming Languages: Python, Node.js — deep expertise in LLM prompt engineering, RAG, agentic workflows, MCP, observability systems. Background in security architecture and log anomaly detection.

School: KTH Royal Institute of Technology — MSc in Information and Communication Systems Security + BSc in Computer and Security

---

Opper AI — Company Profile

Unified AI Gateway and Agent Control Plane for developers. Access 200+ LLMs through a single API with structured output, automatic fallbacks, full observability, evaluations, and agent SDKs (Python & TypeScript). Tagline: *“Make AI infrastructure as reliable as Stripe is for payments.”*

- **Founded:** Oct 2023, launched from stealth Apr 2024
- **Total raised:** ~$3M pre-seed (oversubscribed)
- **Investors:** Luminar Ventures (lead), Emblem VC, Greens Capital, angels
- **Acquisition:** FinetuneDB (Sep 2025) — dataset curation + LLM fine-tuning
- **EU hosting:** Partners with Berget AI for GDPR-compliant, Sweden/EU-hosted inference
- **Products:** Opper API, Opperator (autonomous web agent), Model Interaction Protocol (MIP)

Promptware

Firmware + Program + Memory + Tools + Reflection

github.com/nielsbosma/Promptware